

# TOI 18<sup>th</sup> at CMU

Report from Scientific Committee

# Contest Report

# Incident Day 1

- 09:10 เริ่มทำการสอบ
- มีผู้เข้าแข่งขัน 3 คนที่ โปรแกรม crash ระหว่างการสอบ
  - มีการเปลี่ยนเครื่อง และต่อเวลาให้ 5 นาที 2 คน และ 10 นาที 1 คน
  - ทุกคนใช้ Code::Blocks
  - ในปีถัด ๆ ไป ขอแนะนำให้ผู้เข้าแข่งขันหลีกเลี่ยงการใช้ Code::Blocks (มีแนวโน้มว่าจะไม่ต่อเวลาให้หากเกิดปัญหาจาก code::blocks)
- มีการ “เขียนส่วนของโปรแกรมไว้ล่วงหน้าก่อนเริ่มทำการสอบ”
  - ใน Day 2 จะหักเวลาทำการสอบ

## Incident Day 2

- 09:02 เริ่มทำการสอบ
- 11:10 เกิดอุบัติเหตุมีการส่งตรวจทุก submission
- มีผู้เข้าแข่งขัน หลายคน ที่ โปรแกรม crash ระหว่างการสอบ
  - ฝ่ายควบคุมการสอบแก้ไขปัญหาได้ภายใน 1-2 นาทีในแต่ละคน
  - พิจารณาไม่ต่อเวลาให้
- ไม่มีผู้ใดได้รับโทษตัดเวลาการแข่งขันจากการพิมพ์ส่วนของโปรแกรมไว้มาก่อน

# PROBLEM REPORT

## การเปลี่ยนแปลงแนวทางข้อสอบ

- พยายามยกเลิกการมีข้อง่ายที่วัดความรู้เบื้องต้น ย้ายการวัดความรู้เบื้องต้นไปยัง subtask ง่ายของแต่ละข้อแทน
- มีการจัดกลุ่ม Testcase ทุกข้อ ทำให้ Solution ที่จะได้คะแนนจะต้องผ่านทั้งหมดจริง ๆ ลดปัญหาคะแนนกว้างจากการตรวจซ้ำ
- ขยายแนวทางปีที่แล้วให้มากขึ้น กล่าวคือ เพิ่ม Subtask ที่ “แนะนำ” แนวทางในการแก้ไขปัญหาแทนที่จะมีเพียง Subtask ที่แบ่งตามประสิทธิภาพในการทำงาน
  - ไม่ได้ทำให้ผู้แข่งขันทำข้อสอบได้ลำบากขึ้น เนื่องจากใช้วิธีคิดคะแนน Sum of Subtask
  - รับประกันว่ามี Solution ที่ส่งครั้งเดียวผ่านทุก ๆ subtask

## แนวทางในการทำข้อสอบ

- ถ้าคิด full solution ไม่ได้ อยากรู้ให้พยายามทำทีละ subtask

| Time    | Task                          | Status                  |
|---------|-------------------------------|-------------------------|
| 03:12:4 | <a href="#">d2p1 shopping</a> | ▶ Scored (0.0 / 100.0)  |
| 02:58:2 | <a href="#">d2p1 shopping</a> | ▶ Scored (17.0 / 100.0) |
| 02:25:1 | <a href="#">d2p1 shopping</a> | ▶ Scored (8.0 / 100.0)  |
| 02:15:0 | <a href="#">d2p1 shopping</a> | ▶ Scored (13.0 / 100.0) |
| 02:13:5 | <a href="#">d2p1 shopping</a> | ▶ Scored (13.0 / 100.0) |
| 02:13:4 | <a href="#">d2p1 shopping</a> | ▶ Compilation failed    |

Subtask 3

Subtask 2

Subtask 1

ได้คะแนนรวม 38

## Solution Count

- Number of official solution written by different member of the committee
- Number of official subtask solution

| Problem           | Official Solution | Subtask solution |
|-------------------|-------------------|------------------|
| D1: Mountain      | 4                 | 11               |
| D1: Garden        | 3                 | 6                |
| D1: Constellation | 2                 | 7                |
| D2: Shopping      | 4                 | 10               |
| D2: Sausage       | 4                 | 11               |
| D2: Gecko         | 4                 | 5                |

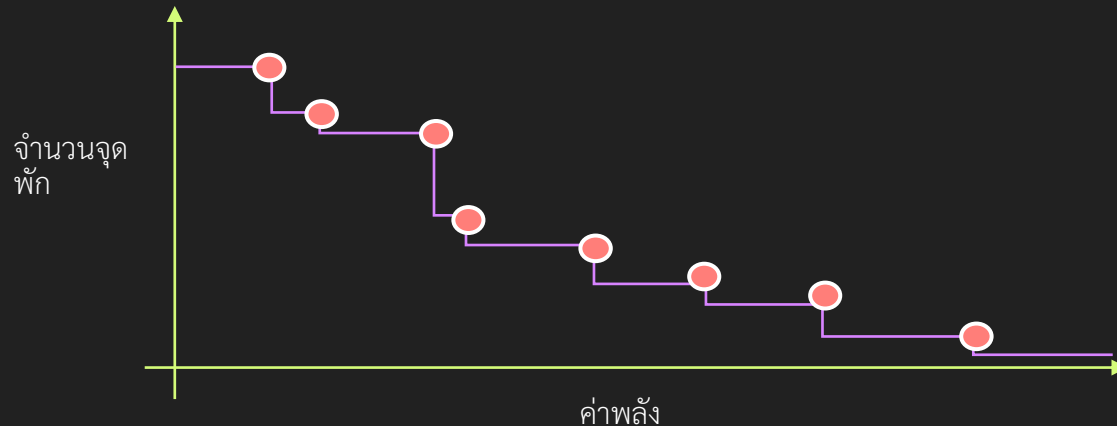
# Mountain Solution

- Official Solution 1  $O(N^3)$

- Dynamic Programming
- สร้าง  $DP[a][b]$  โดย  $DP[a][b]$  คือ “ค่าพลังงานน้อยสุดที่ทำให้เดินไปยังจุดพักหมายเลข  $b$  ได้ โดยแวะผ่าน  $a$  จุด”
- คำนวณโดย  $DP[1][i] = \text{distance}(\text{จุดเริ่ม}, \text{จุดพัก } i)$
- คำนวณโดย  $DP[x][i] = \min( DP[x-1][k], \text{distance}(\text{จุดพัก } k, \text{จุดพัก } i) )$ 
  - $\text{Distance}(p,q)$  คือระยะห่างระหว่างจุดพัก  $p$  กับ  $q$

## Official Solution 2 $O(N^3 \lg(\text{ระยะห่างมากที่สุด}))$

- ให้  $F(x)$  คือฟังก์ชันที่คืนค่า “จำนวนจุดพักที่ต้องใช้” เมื่อนักเดินทางมีพลังเป็น  $x$ 
  - Domain คือ 1..ระยะห่างมากที่สุด  $c$ —domain ของฟังก์ชันคือค่า 1.. $N$
  - สำหรับค่า  $X$  แต่ละค่า เราสามารถคำนวณได้ใน  $O(N^2 \lg N)$  โดยใช้ Dijkstra
- การคำนวณ  $F(X)$  โดยตรงสำหรับค่าพลังของนักเดินทางแต่ละคน คือ partial solution หนึ่ง
- จะเห็นว่า  $F(X)$  เป็น decreasing function ตามค่า  $x$  โดยมี co-domain ที่แตกต่างกันเพียงแค่  $N$  ค่า
  - ใช้ Binary Search ในการหาจุดเปลี่ยนเหล่านั้น (จุดสีแดงในรูปข้างล่าง)
  - สำหรับค่าพลังแต่ละค่า ใช้ lower\_bound เพื่อหาว่า  $F(X)$  ดังกล่าวเป็นเท่าไร



## Mountain Subtask

- (12 คะแนน)  $n \leq 10$ ,  $m \leq 12$ ,  $0 \leq x, y \leq 1000$  (ทำ permutation)
- (16 คะแนน)  $n \leq 250$ ,  $m \leq 400$  (ทำ shortest path ด้วย BFS)
- (7 คะแนน)  $n \leq 100$ ,  $y = 0$  เสมอ (กลายเป็นโจทย์ 1 มิติ ทำให้สามารถเก็บตำแหน่งของจุดพัก แล้วใช้ Binary Search ในการหาจุดพักไกลสุดที่ได้)
- (14 คะแนน)  $n \leq 100$ , ค่า  $s_i$  เรียงจากน้อยไปมาก (ต้องการจะบ่งชี้ให้เห็นว่าคำตอบมันเป็น decreasing function แล้วหวังว่าผู้แข่งขันจะมองออกว่าต้องใช้ Binary Search)
- (9 คะแนน)  $n \leq 200$ ,  $0 \leq x, y \leq 10^8$  (Binary Search)
- (42 คะแนน) (ไม่มีข้อกำหนดอื่นใด) (Binary Search, DP)

# Garden

- Official Solution  $O(n \lg n + Q)$ 
  - คำนวณความยาวของ Longest Increasing Subsequence
  - ให้  $LIS[x]$  คือ ความยาวของ Longest Increasing Subsequence ที่รวมสมาชิกตัวที่  $x$
  - ให้  $LDS[x]$  คือ ความยาวของ Longest Decreasing Subsequence ที่รวมสมาชิกตัวที่  $x$ 
    - คำนวณจาก  $LIS[x]$  โดยวิ่งจาก  $n-1$  ถึง  $0$  แทน
  - สามารถคำนวณ  $LIS$ ,  $LDS$  ได้ใน  $O(n \lg n)$
  - คำตอบของแต่ละค่า  $x$  คือ  $\min(LIS[x], LDS[x])$

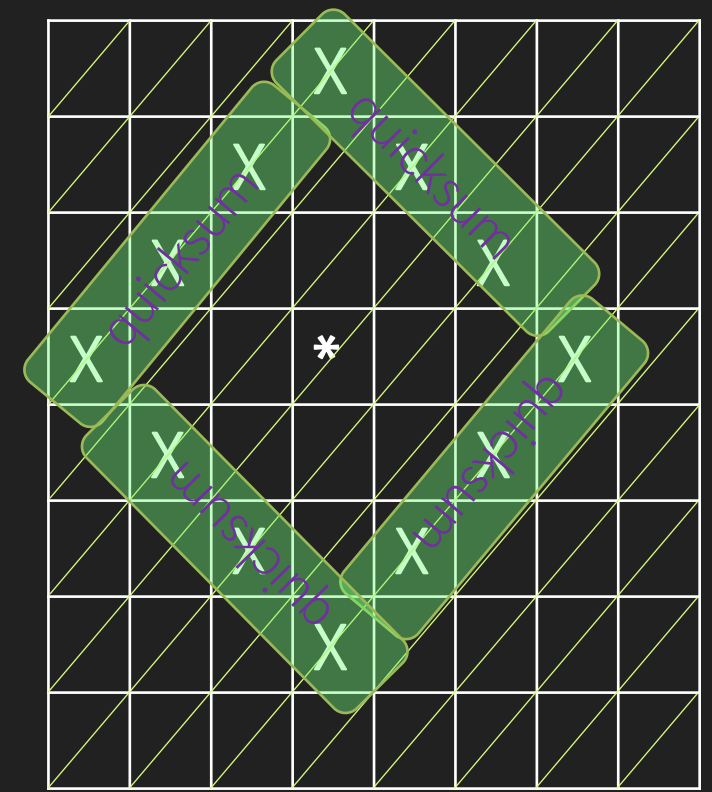
## Garden subtask

- (7 คะแนน)  $N \leq 15$  และ  $A$  ไม่มีค่าซ้ำกัน (Combination)
- (11 คะแนน)  $N \leq 15$  (Combination)
- (14 คะแนน)  $A$  distinct และเป็น unimodal
  - หา peak ให้เจอ ถ้า  $X$  มากกว่า peak เราจะได้รู้ว่าทางขวาของ  $X$  เป็น decreasing sequence แน่นอน ให้ไปหาค่า  $p$  ที่ทำให้  $A[p] < A[x]$  และ  $p < x$  แล้วดูว่า  $P$  กับ  $n-X$  ตัวไหนมากกว่าตอบตัวนั้น (ถ้า  $X$  น้อยกว่า peak ก็ทำกลับด้านกัน)
  - หาค่าแห่งดังกล่าวโดยใช้ binary search (ใช้ lower\_bound )
- (27 คะแนน)  $A[i] < A[i+1]$  สำหรับ  $i \leq x$  แน่ ๆ
  - หา longest decreasing subsequence ที่เริ่มที่  $X$
  - ใ้บอกว่าต้องใช้ Longest decreasing subsequence
- (22 คะแนน)  $n \leq 10,000$ 
  - (ทำเหมือน official sol แต่เขียน Longest Increasing Subsequence แบบ  $O(N^2)$ )
- (19 คะแนน) ไม่มีข้อกำหนดอื่นใด

# Constellation

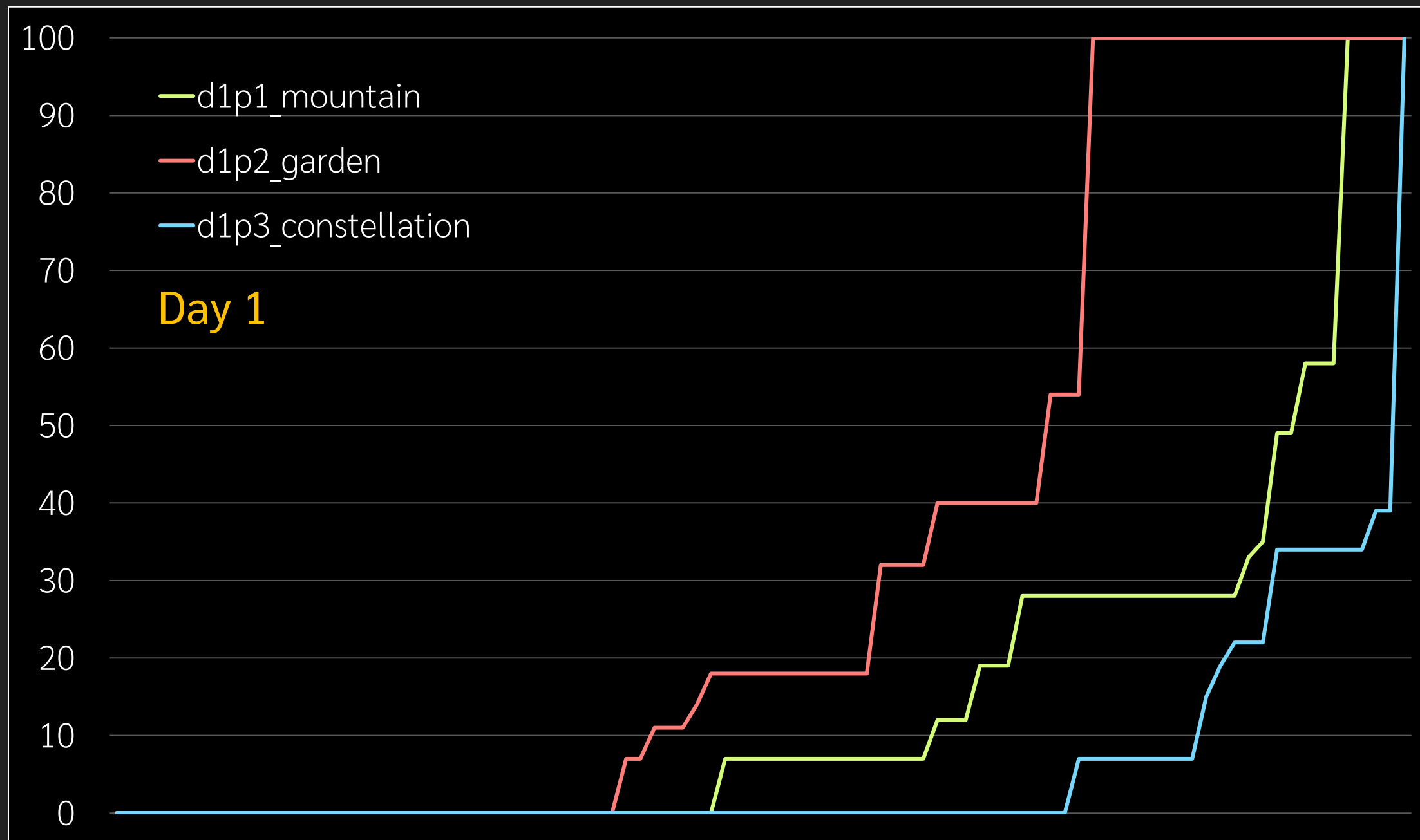
- Official Solution  $O(R \cdot C \cdot (R + C))$

- สร้างตารางขนาด  $R \cdot C$  โดยที่ช่อง  $[r][c]$  มีค่าเป็น 1 เมื่อมีดาวอยู่ในพิกัด  $[r][c]$
- ทำ Quicksum แนวทแยงมุม (ทั้งทแยงขึ้น และ ทแยงลง)
- เพื่อให้ตอบคำถามว่า
  - “ผลรวมของดาว” ตั้งแต่พิกัด  $(r, c)$  ถึง  $(r + k, c + k)$  เป็นเท่าใดได้ใน  $O(1)$
  - “ผลรวมของดาว” ตั้งแต่พิกัด  $(r, c)$  ถึง  $(r + k, c - k)$  เป็นเท่าใดได้ใน  $O(1)$
- จากการตอบคำถามข้างต้นนั้น สำหรับแต่ละพิกัดในตาราง เราสามารถตอบได้ว่ามีดาวอยู่กี่ดวงที่ห่างจากพิกัด  $x$  เป็นระยะทาง  $k$  ได้ใน  $O(1)$  จากการถาม 4 ครั้ง



## Constellation Subtask

- (15 คะแนน) :  $R$  และ  $C \leq 8$  (ให้ทำ simple for loop)
- (12 คะแนน) : จำนวนดาวน้อยกว่าหรือเท่ากับ 18 และ  $k \leq 5$ 
  - ให้ทำ Combination เลือกดาว แล้วหาจุดที่ระยะห่างจากดาวที่เลือกเท่ากัน
- (7 คะแนน) :  $R = 1$  และ  $K = 2$ 
  - กลายเป็นโจทย์ 1 มิติ
  - (เพื่อใบ้ว่าถ้าเรานับได้ว่ามีดาวอยู่ที่ตัวที่ระยะห่าง  $D$  ได้เร็ว ๆ ก็น่าจะผ่าน)
- (5 คะแนน) : ช่องที่มี # นั้นจะอยู่บนพิกัด  $r = c$  เท่านั้น
  - เพื่อใบ้เรื่องการคิดแบบ 45 องศา
- (61 คะแนน) : ไม่มีข้อกำหนดอื่นใด



## Score Summary – Day 1

| โจทย์         | คนเต็ม | คนได้คะแนน | คะแนนรวม |
|---------------|--------|------------|----------|
| Mountain      | 5      | 49         | 1,486    |
| Garden        | 23     | 56         | 3,187    |
| Constellation | 1      | 24         | 579      |

มีคนไม่ได้คะแนน 20 คน  
(ไม่มา 1 คน)

# Shopping

- Official Solution  $O(N + M \lg(N))$ 
  - ทำ Quicksum สองชุด
    - ชุดแรกเป็น Quicksum ของค่าติดลบ ชุดสองเป็น Quicksum ของค่าไม่ลบ
  - สำหรับค่า  $x$   $h$  ที่ให้มา ใช้ Binary Search บน quicksum ชุดแรกเพื่อหา “ตำแหน่งที่ทำให้ผลรวมค่าติดลบจากช่อง  $x$  นั้นมากกว่าหรือเท่ากับพลังของนักช้อป
  - แล้วใช้ Quicksum ชุดที่ 2 ในการหาผลรวมของคะแนนที่ได้
- Naive Solution  $O(NM)$ 
  - ทำตรง ๆ เลย

# Shopping Subtask

- (13 คะแนน) :  $n, m < 1000$ 
  - (for loop ตรง ๆ)
- (8 คะแนน) : รับประกันว่าไม่มี  $A[i]$  ที่ติดลบ
  - ต้องการให้ใช้ Quick sum ตรง ๆ
- (21 คะแนน) :  $h = 1$  เสมอ (เหยียบระเบิดครั้งแรกจะพัง)
  - ต้องการให้จำตำแหน่งของระเบิด แล้วหาให้ได้เร็ว ๆ ว่าจากจุดเริ่มต้นดังกล่าว จะไปหยุดที่จุดใด (ใช้ set มาช่วย จำแล้วใช้ `set::lower_bound`) แล้วขยายผลจาก Quicksum ใน subtask ที่แล้ว
- (17 คะแนน) :  $x = 0$  เสมอ
  - ให้หา sum ของช่องติดลบที่ไม่น้อยกว่า  $h$  แล้วหาช่องที่  $sum \geq h$  เร็ว ๆ ทำได้โดยใช้ `map::lower_bound`
- (41 คะแนน): ไม่มีข้อบังคับอื่นใด

## Sausage Official Solution $O(N^2)$

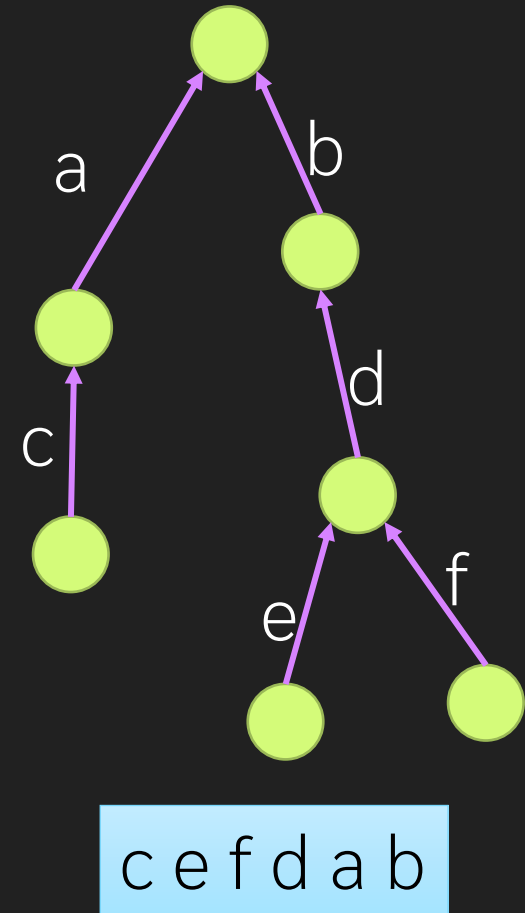
- ทำ Dynamic Programming 2 ชั้น
- ชั้นแรกเป็น DP 2 มิติ
  - ให้  $eat[L][R]$  เป็นค่าความอร่อยรวมมากที่สุด เมื่อพิจารณาเฉพาะไส้อ้วน ณ ช่องหมายเลข  $L$  ถึง  $R$  เท่านั้น โดยไม่มีการตัดสายไส้อ้วน
  - $eat[L][L] = A[L]$
  - $eat[L][R] = \max(A[L] + eat[L+1][R], A[R] + eat[L][R-1]) + |A[L] - A[R]|$
- ชั้นสองเป็น DP 1 มิติ
  - ให้  $DP[i]$  คือค่าความอร่อยรวมมากที่สุดเมื่อพิจารณาเฉพาะไส้อ้วน ณ ช่องหมายเลข 1 ถึง  $i$  โดยสามารถตัดไส้อ้วนได้
  - $DP[i] = \max(DP[L] + eat[L+1][i])$  สำหรับ  $L$  ตั้งแต่ 0 ถึง  $i-1$

## Sausage Subtask

- (13 คะแนน) :  $n < 20$  (recursive simulate)
- (12 คะแนน) :  $n < 100$  รับประกันว่าคำตอบที่ได้ ไม่ได้เกิดจากการแบ่งไส้อ้วน
  - เพื่อให้ทำ dynamic programming ชั้นแรก
- (13 คะแนน) : รับประกันว่าคำตอบที่ได้ เกิดจากการแบ่งไส้อ้วน ไม่เกิน 4 ก้อน  $n < 100$  สำหรับ
  - ใ้เห็นถึงการแยกส่วนกันระหว่าง DP ชั้นแรกกับการตัดแบ่ง  $O(N^4)$
- (11 คะแนน) :  $N \leq 500$ 
  - เพื่อให้ต่อยอดมา Dynamic programming  $O(N^3)$
- (16 คะแนน) : รับประกันว่าคำตอบที่ได้ เกิดจากการแบ่งไส้อ้วน ไม่เกิน 10 ก้อน
  - $n < 1000$  สำหรับ recursive การตัดแบ่ง dynamic  $O(KN^2)$
- (35 คะแนน): ไม่มีข้อบังคับอื่นใด  $O(N^2)$

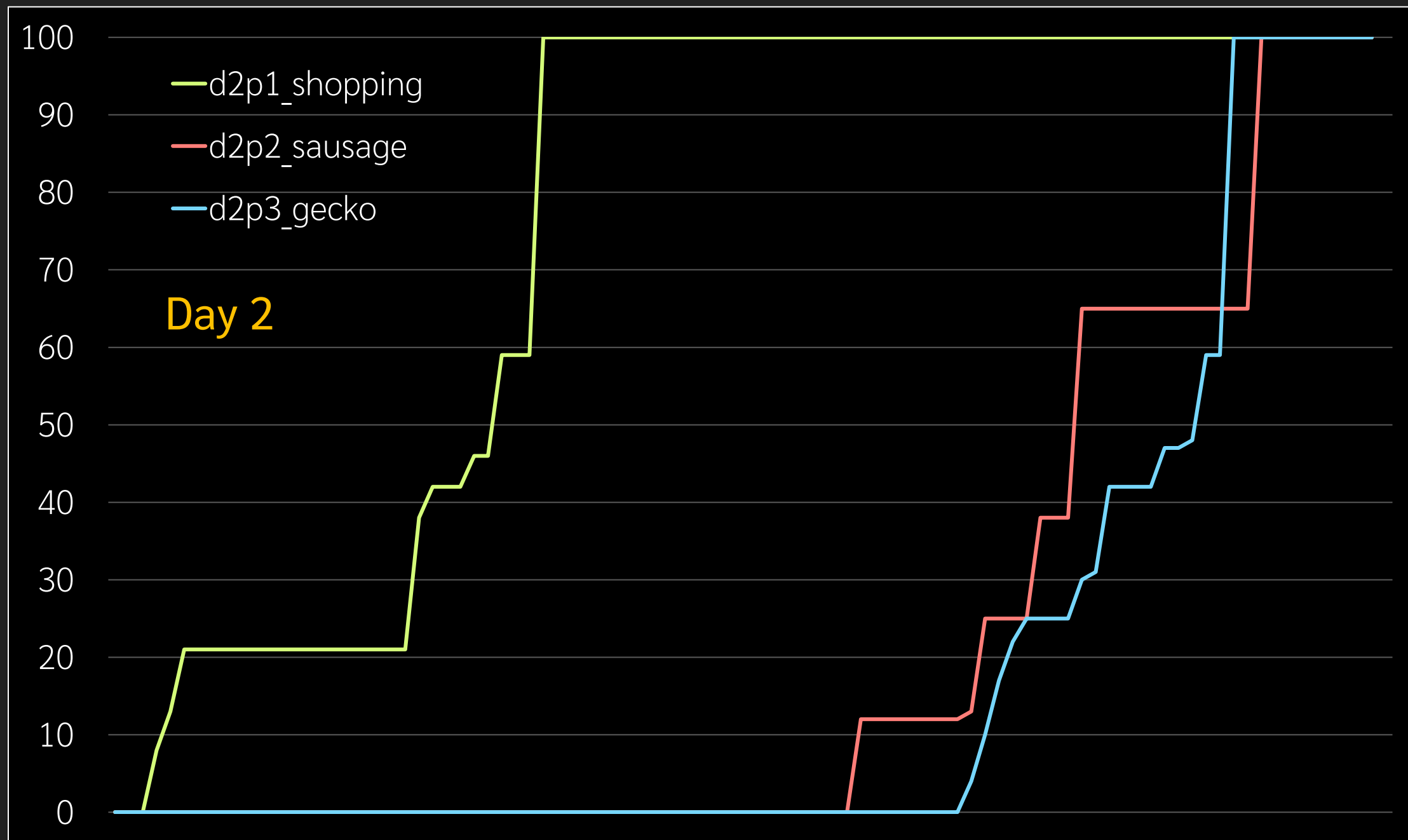
## Gecko Official Solution $O(M \lg N)$

- แปลงเป็นกราฟ ซึ่งหนึ่งคือ node ไม้ยัดขวางคือ เส้นเชื่อม
  - เส้นเชื่อมแบบไม่มีทิศทาง
- หา shortest path บนกราฟแบบ multiple source (ปมจิ้งจก) โดยคิด shortest path จากปลายทาง (ปมแมงมุม)
- ผลลัพธ์ที่ได้จะเป็น tree แล้วต้องแปลง tree นั้นกลับมาเป็นเส้นเชื่อมให้ตรงตามวิธีการเดินของจิ้งจก
- สังเกตว่า tree นั้นมี root เป็นปลายทาง และมี leaves เป็นต้นทาง
  - ให้คิดว่า tree เป็น directed tree ที่ทิศทางของเส้นเชื่อมชี้จาก leave ไปยัง root
  - จิ้งจกเลือกเดินไม้ยัดต่ำสุด แปลว่า ต้องให้เส้นเชื่อมที่ออกจากปมไปยังรากนั้นอยู่ด้านล่าง
  - ทำ topological sorting บน directed tree (สามารถทำได้ตรง ๆ จาก DFS หรือ BFS ก็ได้)



## Geck subtask

- (17 คะแนน) :  $k < n \leq 10$ ,  $m \leq 10$  (ทำ permutation ของเส้นเชื่อมตรง ๆ)
- (4 คะแนน) :  $k = 1$ ,  $w = 1$ , รับประกันว่ามีคำตอบที่ไม่ต้องเรียงกิ่ง
  - ใ้ว่าต้องแปลงเป็นกราฟแล้วหา path ให้เจอ
- (6 คะแนน) :  $w = 1$ , รับประกันว่ามีคำตอบที่ไม่ต้องเรียงกิ่ง
  - ใ้ว่าต้องการ shortest path บน unweighted graph (ใช้ BFS ได้)
- (9 คะแนน) :  $k = 1$ , รับประกันว่ามีคำตอบที่ไม่ต้องเรียงกิ่ง (หา shortest path)
  - ใ้เขียน Shortest Path
- (12 คะแนน) : รับประกันว่ามีคำตอบที่ไม่ต้องเรียงกิ่ง (หา shortest path tree)
  - ใ้เขียน Shortest Path แบบ multiple source
- (11 คะแนน) :  $w = 1$ 
  - ต้องเรียงกิ่งแล้ว
- (41 คะแนน): ไม่มีข้อบังคับอื่นใด

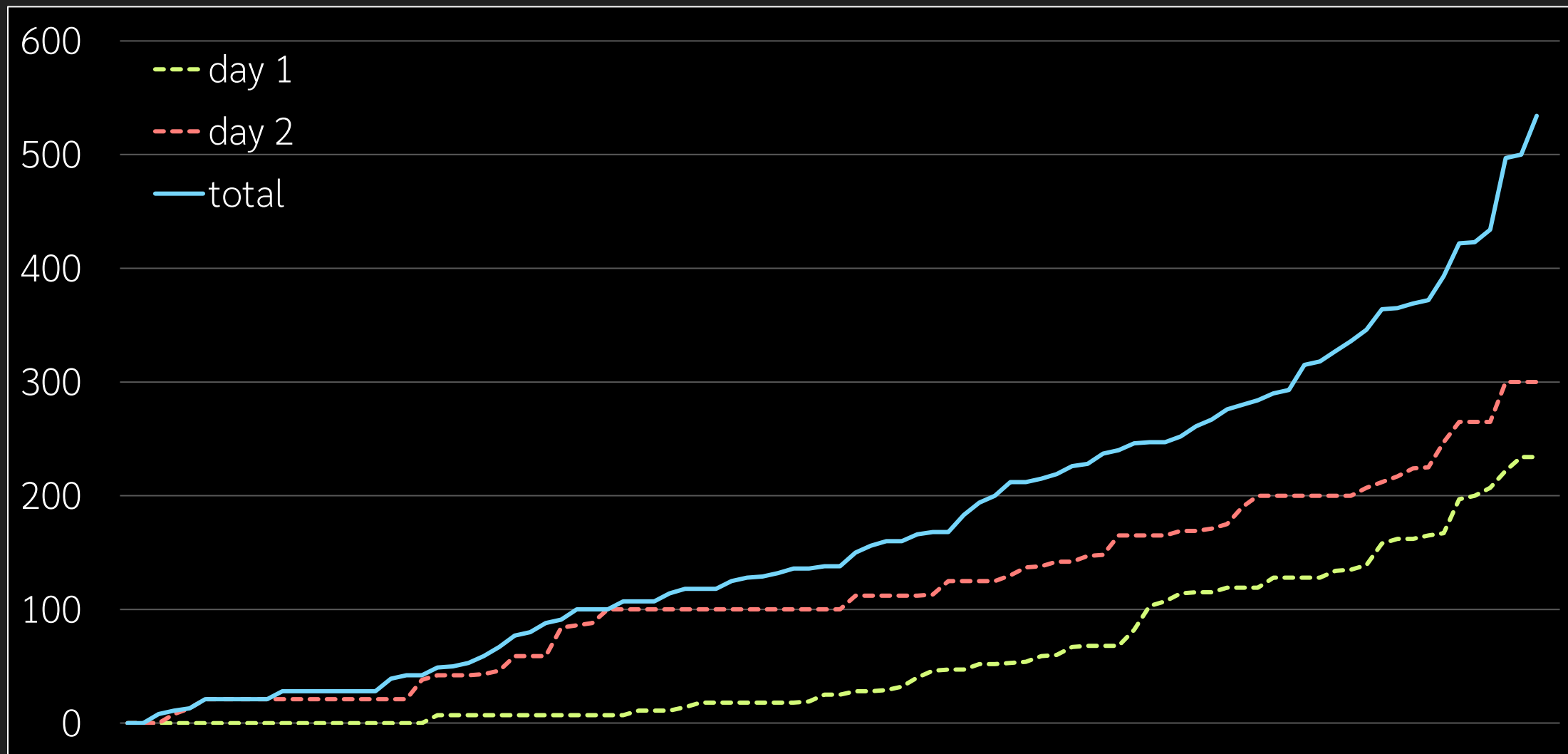


## Score Summary – Day 2

| โจทย์    | คนเต็ม | คนได้คะแนน | คะแนนรวม |
|----------|--------|------------|----------|
| Shopping | 61     | 89         | 6,911    |
| Sausage  | 9      | 38         | 2,068    |
| Gecko    | 11     | 30         | 1,742    |

มีคนไม่ได้คะแนน 3 คน  
(ไม่มา 1 คน)

## TOI 18 Summary



# Medal Allocation

## Medal Allocation

- S5.10 ก่อนพิธีมอบรางวัล TOI นั้น SC ต้องยื่นเรื่องต่อ GA เกี่ยวกับการมอบเหรียญรางวัลให้ผู้เข้าแข่งขันตามคะแนนที่ผู้เข้าแข่งขันได้รับ GA ต้องยืนยันคะแนนและรางวัลของผู้เข้าแข่งขัน ผู้เข้าแข่งขันไม่เกินครึ่งหนึ่งจะได้รับเหรียญรางวัลตามหลักดังนี้
  - ผู้เข้าแข่งขันประมาณหนึ่งในสิบสองของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญทอง
  - ผู้เข้าแข่งขันประมาณหนึ่งในหกของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญเงิน
  - ผู้เข้าแข่งขันประมาณหนึ่งในสี่ของผู้เข้าแข่งขันทั้งหมดได้รับเหรียญทองแดง
- GA ต้องยืนยันข้อเสนอในการจัดสรรเหรียญรางวัลก่อนให้คณะผู้แทนทั้งหมดรับทราบผลคะแนน

# Competitor

- รายชื่อทั้งหมดมีนักเรียน 92 คน
  - Fossil 3 คน
  - ไม่มาสอบ 1 คน
- จำนวนคนที่ใช้คิดในการตัดแบ่งเหรียญรางวัล  $= 92 - 3 - 1 = 88$  คน

# Medal Allocation Proposal

- การคำนวณเหรียญจะใช้จำนวนคน 88 คน
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย  $88/12 = 7.5 \sim 8$  คนได้เหรียญทอง
- เกณฑ์คะแนนขั้นต่ำของเหรียญเงินคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย  $88/6 = 15$  คนได้เหรียญเงิน
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองแดงคือคะแนนต่ำสุดที่ทำให้ไม่เกิน  $88/2 = 44$  คนได้เหรียญใดเหรียญหนึ่ง แปลว่าต้องมีทองแดง 21 คน (น้อยกว่าปีที่แล้ว 1 คน)
  - เนื่องจาก  $8+15+21 = 45$
  - เหรียญทองสุดท้าย – เหรียญเงินแรก = 5 คะแนน
  - เหรียญเงินสุดท้าย – เหรียญทองแดงแรก = 1 คะแนน
  - เหรียญทองสุดท้าย – เหรียญทองแดงแรก = 123 คะแนน

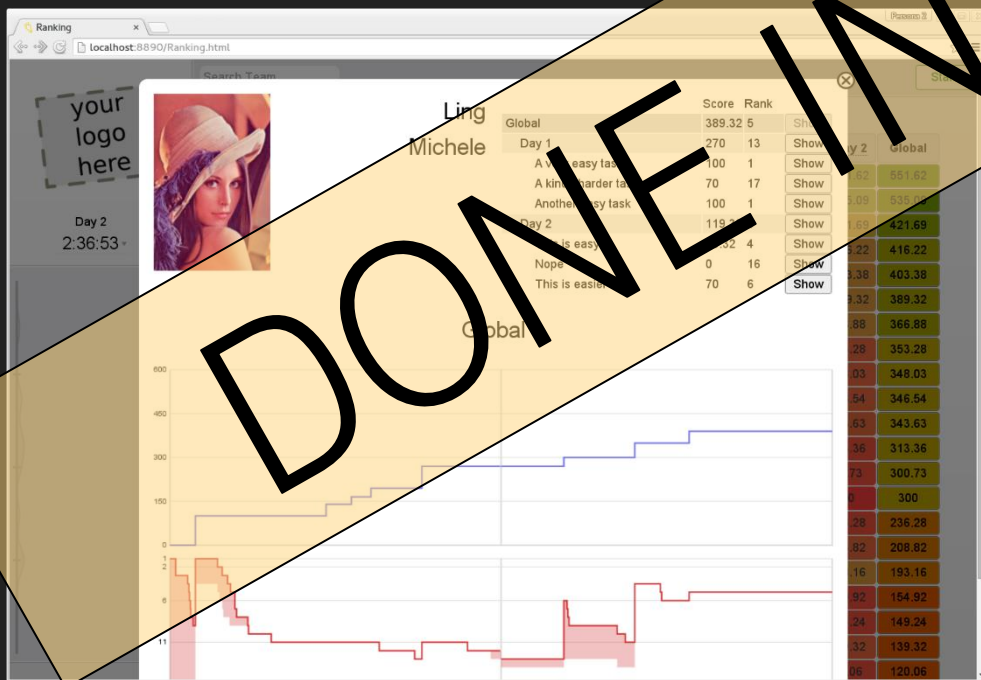
## Medal Allocation Proposal Option 2

- การคำนวณเหรียญจะใช้จำนวนคน 90 คน (ยึดตามโควตาที่เป็นไปได้)
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย  $88/12 = 7.5 \sim 8$  คนได้เหรียญทอง (ไม่เปลี่ยนแปลงจาก option 1)
- เกณฑ์คะแนนขั้นต่ำของเหรียญเงินคือคะแนนสูงสุดที่ทำให้มีอย่างน้อย  $88/6 = 15$  คนได้ เหรียญเงิน (ไม่เปลี่ยนแปลงจาก option 1)
- เกณฑ์คะแนนขั้นต่ำของเหรียญทองแดงคือคะแนนต่ำสุดที่ทำให้ไม่เกิน  $88/2 = 45$  คนได้เหรียญใดเหรียญหนึ่ง แปลว่าต้องมีทองแดงไม่เกิน 22 คน
  - ทำไม่ได้ เนื่องจาก ทองแดงอันดับที่ 22 และ 23 ได้คะแนนเท่ากัน ทำให้ต้องมีทองแดง 23 คน ซึ่งเกินครึ่งหนึ่ง
- ยังคงต้องให้ทองแดง 21 คน (ไม่เปลี่ยนแปลงจาก option 1)
- ถ้าจะให้ 23 คน GA ต้อง VOTE เพื่อรับรอง (ไม่ตรงกับธรรมนูญ แต่ทำได้)

# Issue and Improvement

# Proposal from the last year

- ปัญหาของ sequel ส่วนหนึ่งเกิดมาจาก grader ไม่รองรับความต้องการที่เฉพาะเจาะจงของโจทย์
- grader ปัจจุบันไม่ตอบโจทย์ในหลายเครื่อง
- ioi ใช้ CMS สำหรับการแข่งขันมาหลายปีแล้ว



My contest

Server time: 00:00:43

Time left: 02:51:16

[Overview](#)
[Communication](#)
[MYTASK](#)
[Statement](#)
[Submissions](#)

[MYOHTERTASK](#)
[Statement](#)
[Submissions](#)

[YOUGETIT](#)
[Statement](#)
[Submissions](#)

[Documentation](#)
[Testing](#)

Persons 2

localhost:8080/task/compression/description

My contest

Logged in as Mario Rossi (m.rossi)

Logout Automatic

A very hard example task (mytask)

description

Statement

Download task statement

Some details

|                      |                                        |                                                                                                   |
|----------------------|----------------------------------------|---------------------------------------------------------------------------------------------------|
| Type                 | Two steps                              |                                                                                                   |
| Time limit           | 1 second                               |                                                                                                   |
| Memory limit         | 256 MiB                                |                                                                                                   |
| Compilation commands | C                                      | /usr/bin/gcc -DEVAL -static -O3 -std=c11 -o manager mytask_send.c mytask_recv.c manager.c -lm     |
|                      | C++                                    | /usr/bin/g++ -DEVAL -static -O3 -std=c++11 -o manager mytask_send.cpp mytask_recv.cpp manager.cpp |
| Tokens               | You have an infinite number of tokens. |                                                                                                   |

Attachments

|                                                                        |                                                                        |                                                                      |
|------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------|
| <div>mytask_recv.c</div> <div>C source code</div> <div>646 B</div>     | <div>mytask_send.c</div> <div>C source code</div> <div>505 B</div>     | <div>grader.c</div> <div>C source code</div> <div>1.97 KiB</div>     |
| <div>mytask_recv.cpp</div> <div>C++ source code</div> <div>646 B</div> | <div>mytask_send.cpp</div> <div>C++ source code</div> <div>505 B</div> | <div>grader.cpp</div> <div>C++ source code</div> <div>1.97 KiB</div> |

Contest Management System is released under the GNU Affero General Public License .

# Testcase generation by Polygon

- ทดสอบความแข็งแรงของ test data โดยใช้ Polygon
- เริ่มทำแล้วในปีนี้เป็นบางข้อ (ข้อ sequel ก็ทำ.... แต่ไม่ได้ทำตอนที่รันตัด testcase)

| #  | 1_TransitiveClosure_1221.cpp | 2_Dijkstra_BB_DP_1221.cpp | 3_Dijkstra_BB_1221.cpp  | 4_Dijkstra_1221.cpp     | 5_GreedyDP_1221_noop0.cpp | 6_GreedyDP_1221_BFS_noVisitedNode.cpp | 7_Backtracking_1221.cpp | 8_DFS_1221.cpp        |
|----|------------------------------|---------------------------|-------------------------|-------------------------|---------------------------|---------------------------------------|-------------------------|-----------------------|
| 1  | OK 0 / 5 / 5.00000           | OK 0 / 9 / 5.00000        | OK 15 / 6 / 5.00000     | OK 0 / 6 / 5.00000      | OK 15 / 6 / 5.00000       | OK 0 / 6 / 5.00000                    | OK 15 / 9 / 5.00000     | OK 0 / 6 / 5.00000    |
| 2  | OK 0 / 5 / 5.00000           | WA 0 / 9 / 5.00000        | OK 15 / 6 / 5.00000     | OK 0 / 6 / 5.00000      | OK 0 / 6 / 5.00000        | OK 654 / 78 / 5.00000                 | OK 0 / 9 / 5.00000      | OK 186 / 6 / 5.00000  |
| 3  | OK 0 / 5 / 5.00000           | WA 15 / 9 / 5.00000       | OK 0 / 6 / 5.00000      | OK 15 / 6 / 5.00000     | OK 15 / 6 / 5.00000       | OK 15 / 6 / 5.00000                   | OK 15 / 9 / 5.00000     | OK 15 / 6 / 5.00000   |
| 4  | OK 15 / 5 / 5.00000          | WA 15 / 9 / 5.00000       | OK 0 / 6 / 5.00000      | OK 15 / 6 / 5.00000     | OK 15 / 6 / 5.00000       | OK 0 / 6 / 5.00000                    | OK 0 / 9 / 5.00000      | OK 15 / 6 / 5.00000   |
| 5  | OK 15 / 5 / 5.00000          | WA 0 / 9 / 5.00000        | OK 46 / 6 / 5.00000     | OK 46 / 6 / 5.00000     | OK 93 / 6 / 5.00000       | ML 483 / 256 / 5.00000                | OK 343 / 9 / 5.00000    | TL 1000 / 6 / 5.00000 |
| 6  | OK 15 / 5 / 5.00000          | TL 1000 / 15 / 5.00000    | TL 1000 / 25 / 5.00000  | TL 1000 / 15 / 5.00000  | TL 1000 / 150 / 5.00000   | ML 499 / 256 / 5.00000                | TL 1000 / 9 / 5.00000   | TL 1000 / 6 / 5.00000 |
| 7  | OK 15 / 5 / 5.00000          | WA 0 / 9 / 5.00000        | OK 62 / 6 / 5.00000     | OK 93 / 6 / 5.00000     | OK 124 / 6 / 5.00000      | ML 529 / 256 / 5.00000                | OK 62 / 9 / 5.00000     | TL 1000 / 6 / 5.00000 |
| 8  | OK 61 / 5 / 5.00000          | TL 514 / 256 / 5.00000    | ML 498 / 256 / 5.00000  | ML 499 / 256 / 5.00000  | ML 405 / 256 / 5.00000    | ML 436 / 256 / 5.00000                | TL 1000 / 10 / 5.00000  | TL 1000 / 7 / 5.00000 |
| 9  | OK 15 / 5 / 5.00000          | WA 0 / 9 / 5.00000        | OK 826 / 8 / 5.00000    | TL 1000 / 8 / 5.00000   | TL 1000 / 8 / 5.00000     | ML 499 / 256 / 5.00000                | TL 1000 / 9 / 5.00000   | TL 1000 / 7 / 5.00000 |
| 10 | OK 265 / 5 / 5.00000         | WA 483 / 20 / 5.00000     | TL 1000 / 17 / 5.00000  | TL 1000 / 15 / 5.00000  | TL 1000 / 15 / 5.00000    | ML 530 / 256 / 5.00000                | TL 1000 / 11 / 5.00000  | TL 1000 / 8 / 5.00000 |
| 11 | OK 140 / 5 / 5.00000         | TL 1000 / 83 / 5.00000    | TL 1000 / 80 / 5.00000  | TL 1000 / 78 / 5.00000  | ML 467 / 256 / 5.00000    | ML 436 / 256 / 5.00000                | TL 1000 / 11 / 5.00000  | TL 1000 / 8 / 5.00000 |
| 12 | OK 62 / 5 / 5.00000          | TL 1000 / 47 / 5.00000    | TL 1000 / 152 / 5.00000 | TL 1000 / 150 / 5.00000 | TL 1000 / 150 / 5.00000   | ML 468 / 256 / 5.00000                | TL 1000 / 10 / 5.00000  | TL 1000 / 8 / 5.00000 |
| 13 | OK 295 / 5 / 5.00000         | OK 62 / 16 / 5.00000      | OK 62 / 13 / 5.00000    | OK 46 / 11 / 5.00000    | OK 31 / 11 / 5.00000      | ML 498 / 256 / 5.00000                | OK 124 / 11 / 5.00000   | TL 1000 / 8 / 5.00000 |

จำนวน Solution

# Proposal for TOI 19

- การรันใหม่สามครั้ง
- แนวทางเพิ่มเติมของโจทย์
  - เขียนเฉพาะฟังก์ชัน
  - โจทย์ Interactive

# การรันซ้ำ

- ผลคะแนนของ TOI 18 ไม่มีการเปลี่ยนแปลงจากการรันใหม่สามครั้ง
- Grader ที่ใช้มี stability ค่อนข้างสูง
- โจทย์เน้นไปที่การมัด testcase มากขึ้น
- นักเรียนได้ feedback เกี่ยวกับเวลาที่ใช้โดยละเอียด
- คิดว่าการรันซ้ำไม่มีความจำเป็น มีโอกาสทำให้คะแนนลดลงโดยไม่จำเป็น
- เสนอว่า TOI 19 ลดเหลือการรันซ้ำเพียงครั้งเดียว ส่วน TOI 20 ยกเลิกไปเลย
  - (หรือไม่ก็ยกเลิกไปเลยตั้งแต่ TOI 19)

# การเขียนโปรแกรมเฉพาะฟังก์ชัน

- เป็นแนวทางที่ IOI ใช้
- ผู้เข้าแข่งขันเขียนเฉพาะฟังก์ชันที่จำเป็นต่อการตอบคำถาม
  - ไม่ต้องเขียนการรับค่าอินพุตจาก keyboard หลีกเลียงปัญหาการต้องเขียน IO เช่น `ios_base::sync_with_stdio`
  - ไม่ต้องเขียนการส่งค่าเอาพุตออกหน้าจอ (หลีกเลียงปัญหา `endl`, “`\n`” )
- ข้อเสีย
  - จำเป็นต้องใช้ไฟล์ `.h` ในการเขียนโปรแกรม
  - นักเรียนต้อง compile file หลาย ๆ ไฟล์เป็น
  - คำอธิบายในโจทย์อาจจะยาวขึ้น
- ข้อดี
  - ทำให้สร้างโจทย์แบบ interactive ได้

# โจทย์ Interactive

- แทนที่โจทย์จะเป็นการรับ input ครั้งเดียว แล้วตอบผลครั้งเดียว จะเป็นการเรียกฟังก์ชันที่ grader เตรียมไว้
- เป็นโจทย์ที่โผล่มาเรื่อย ๆ ใน IOI
- ข้อเสีย
  - ค่อนข้างซับซ้อนในการเริ่มทำ

## ตัวอย่างโจทย์

- มีอาร์เรย์  $A[1..n]$  (เราไม่ได้รับข้อมูลค่าของ  $A[i]$  กล่าวคือ เราไม่ได้  $A$  มาเป็น input ของโจทย์ เราทราบเพียงค่า  $n$  ว่ามี  $A[i]$  กี่ช่อง)
  - $A[i]$  แต่ละช่องไม่ซ้ำกัน
- มีฟังก์ชัน `bool check(int p, int q)` ให้เรียกใช้
  - `check(p,q)` จะคืนค่า `true` ก็ต่อเมื่อ  $A[p] > A[q]$
  - เราสามารถเรียก `check` ได้หลาย ๆ ครั้ง
- โปรแกรมจะต้องหาค่า  $u$  และ  $v$  ที่ทำให้  $A[u]$  มากที่สุด และ  $A[v]$  น้อยที่สุด

## ตัวอย่าง

- คะแนน ที่ได้แปรผกผันกับ “จำนวนครั้ง” ที่เรียก check
- เรียกเยอะ ๆ ได้คะแนนน้อย
- เรียกน้อย ๆ ได้คะแนนเยอะ

```
pair<int,int> solve(int n) {  
    int p,q;  
    for (int i = 0;i < n;i++) {  
        bool is_max = true;  
        bool is_min = true;  
        for (int j = i+1;j < n;j++) {  
            if (check(i,j) == false) {  
                is_max = false;  
            } else {  
                is_min = false;  
            }  
        }  
        if (is_max) { p = i; }  
        if (is_min) { q = i; }  
    }  
  
    return make_pair(p,q);  
}
```

# Thank you!

- สรุปมติที่ประชุม 8 ทอง 15 เงิน 23 ทองแดง
- (คำแนะนำจาก GA) ปีหน้าลองพยายามรันซ้ำ 1 ครั้งพอ
  - ทำได้หรือไม่ให้เจ้าภาพเป็นผู้กำหนด
- (คำแนะนำจาก GA) ปีหน้าลองพยายามเขียนแบบส่ง function อย่างเดียวพอ
  - ทำได้หรือไม่ให้เจ้าภาพเป็นผู้กำหนด